

**Dr. D. Y. Patil Institute of Management and Entrepreneur Development
(DYPIMED), Talegaon**

Python Programming

Lecture Notes

Introduction to Python • Data Types and Operators • Conditional Statements

Prepared by

Ajinkya Ban

Contents

1. Introduction to Python	3
1.1 What is Python?	3
1.2 What does “general purpose” mean?	3
1.3 What does “interpreted” mean?	3
1.4 Types of programming languages	4
1.5 Is Python compiled or interpreted?	4
1.6 What is bytecode? Can we see it?	5
1.7 Can we read the bytecode?	5
1.8 What is a dynamically typed language?	6
1.9 Advantages of Python	6
1.10 How to install Python	7
1.11 Different ways to run a Python program	7
1.12 Virtual environment (venv and Miniconda)	8
1.13 Checking code quality with pylint	9
1.14 What is requirements.txt?	9
1.15 Garbage Collector (gc)	10
2. Data Types and Operators	11
2.1 What is a variable?	11
2.2 Ways to give values to variables	11
2.3 What is a data type?	12
2.4 What are operators?	13
3. Conditional Statements (Decision Making)	19
3.1 Simple if	19
3.2 if – else	19
3.3 if – elif – else	20
3.4 Nested if – else	20
3.5 match – case (Python 3.10 and newer)	21
3.6 Keywords in Python	22

1. Introduction to Python

1.1 What is Python?

Definition: Python is a general-purpose, interpreted, high-level, object-oriented, platform-independent, dynamically typed scripting language.

In simple words: Python is a programming language that is easy to read and write. We use it to give instructions to the computer.

Python was created by **Guido van Rossum** in **1991**. Python code looks almost like simple English, so it is a good first language for beginners.

Here is your first Python program. It prints a message on the screen:

EXAMPLE

```
print("Hello, DYPIMED!")
```

OUTPUT

```
Hello, DYPIMED!
```

1.2 What does “general purpose” mean?

General purpose means Python can be used for **many different kinds of work**, not only one. For example, it is not only for Machine Learning.

Area of work	Python libraries / tools used
Websites and web apps	Django, Flask, FastAPI
Data Science and Machine Learning	NumPy, Pandas, scikit-learn, TensorFlow
AI and GenAI	LangChain, Hugging Face
Image processing	OpenCV, Pillow
Desktop apps (with windows and buttons)	Tkinter, PyQt
Mobile apps	Kivy, BeeWare
Automation and testing	Selenium, PyTest

In simple words: Python is like a Swiss knife: one tool that can do many jobs.

1.3 What does “interpreted” mean?

An interpreted language runs the program **one line at a time**, from top to bottom.

If there is a mistake on line 3, then lines 1 and 2 still run. The program stops only when it reaches the wrong line.

CODE

```
print("Line 1 runs")
print("Line 2 runs")
print(10 / 0)          # mistake: we cannot divide by zero
print("Line 4 never runs")
```

OUTPUT

```
Line 1 runs
Line 2 runs
Traceback (most recent call last): ...
ZeroDivisionError: division by zero
```

In simple words: Think of a translator who translates a speech sentence by sentence, while the person is speaking.

1.4 Types of programming languages

Type	What it means	Examples
Procedural	The program is a list of steps (functions)	C, Pascal
Object-Oriented	The program is built using classes and objects	Java, C++, Python
Scripting	Runs line by line; used for automation and websites	JavaScript, PHP, Ruby, Python
Markup	Tells how a page or document is structured	HTML, XML
Stylesheet	Tells how a page looks (colours, fonts)	CSS

Note: Python fits into more than one group. It is object-oriented and also a scripting language.

1.5 Is Python compiled or interpreted?

The answer is **both**. When you run a Python program, two things happen:

1. The **compiler** first converts your code (.py file) into **bytecode** (.pyc file).
2. The **interpreter**, called the **PVM (Python Virtual Machine)**, then runs this bytecode line by line.

Your code (.py) → Compiler → Bytecode (.pyc) → PVM → Output

The PVM is called **the heart of Python**, because it is the part that actually runs your program.

Difference between Compiler and Interpreter

Compiler	Interpreter
Reads the whole program at once	Reads and runs one line at a time
Shows all errors together	Stops at the first error it finds
Program runs faster	Program runs slower

Compiler	Interpreter
In Python: converts .py into .pyc	In Python: the PVM runs the .pyc file

For each line, the interpreter does three small jobs:

1. **Read** the instruction
2. **Understand** the instruction
3. **Run** the instruction

1.6 What is bytecode? Can we see it?

Definition: Bytecode is a set of simple instructions that Python makes from your code. The PVM runs these instructions.

In simple words: Bytecode is the “half-translated” form of your program. It is not your code, and it is not machine code; it is in between.

Bytecode is **machine-independent**. This means the same bytecode can run on Windows, Linux or Mac.

Yes, we can create the bytecode file. First, make a file called **add.py**:

CODE

```
a = 10
b = 20
print(a + b)
```

Then type this command:

COMMAND

```
python -m py_compile add.py
```

Python creates a new folder named **__pycache__**. Inside it you will find a file like **add.cpython-312.pyc**. Here **312** means Python version 3.12.

Part	Meaning
-m	Run a Python module (library)
py_compile	The module that makes the .pyc (bytecode) file

1.7 Can we read the bytecode?

Yes. The **dis** module (short for **disassembler**) shows the bytecode in a form humans can read.

COMMAND

```
python -m dis add.py
```

We can also use it inside a program:

EXAMPLE

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

OUTPUT

```
3          0 RESUME               0

4          2 LOAD_FAST            0 (a)
          4 LOAD_FAST            1 (b)
          6 BINARY_OP             0 (+)
         10 RETURN_VALUE
```

You can read it like this: load a, load b, add them (+), and return the answer.

Note: This output is from Python 3.11. Other versions may show it a little differently.

1.8 What is a dynamically typed language?

In languages like C, C++ and Java, you must tell the **type** of every variable when you create it:

CODE

```
int num = 10;           // C / C++ / Java
float salary = 455.50;
```

In Python, you **do not write the type**. Python looks at the value and decides the type by itself when the program runs. This is called **dynamic typing**.

To check which type Python has given, use the **type()** function:

EXAMPLE

```
num = 10
print(type(num))

num = "ten"           # same variable, now holding text
print(type(num))

salary = 455.50
print(type(salary))
```

OUTPUT

```
<class 'int'>
<class 'str'>
<class 'float'>
```

In simple words: In Python, a variable is like a box with no label. You can put a number in it today and text tomorrow.

1.9 Advantages of Python

1. **Easy to learn** – the code looks like simple English.
2. **Platform independent** – the same code runs on Windows, Linux and Mac.

3. **Dynamically typed** – no need to write data types.
4. **Object-oriented** – supports classes and objects.
5. **Free and open source** – anyone can download and use it for free.
6. **Huge library support** – ready-made code is available for almost everything.
7. **High-level** – Python handles memory by itself.
8. **Can make desktop apps** – with Tkinter or PyQt.
9. **Can make websites** – with Django or Flask.
10. **Big community** – lots of help, videos and tutorials online.

1.10 How to install Python

On Windows

1. Open <https://www.python.org/downloads/> and download the latest Python 3 version (for example, 3.12).
2. Run the installer. **Important: tick the box “Add python.exe to PATH”**, then click Install Now.
3. Open Command Prompt and type the command below to check that Python is installed.

COMMAND

```
python --version
```

OUTPUT

```
Python 3.12.10
```

On Linux (Ubuntu)

COMMAND

```
sudo apt update
sudo apt install python3 python3-pip
python3 --version
```

1.11 Different ways to run a Python program

Tool	What it is	Good for
Notepad + Command Prompt	Write code in Notepad, run it from Command Prompt	Learning the basics
Python Shell (>>>)	Type one line, see the answer at once	Quick testing
IDLE	Simple editor that comes with Python	Beginners
Jupyter Notebook	Write and run code in small blocks (cells)	Data Science, practice
Google Colab	Jupyter in the browser, free from Google	ML work, no installation
PyCharm	Professional code editor (IDE)	Big projects
Online compiler	Run code on a website	When Python is not installed

Tool	What it is	Good for
Anaconda / Miniconda	Python plus extra tools for Data Science	Data Science projects

1. Notepad + Command Prompt

Write your code in Notepad and save it as **hello.py**. Then open Command Prompt in that folder and type:

COMMAND

```
python hello.py
```

2. Python Shell

Type **python** in Command Prompt. You will see **>>>**. Type any line and press Enter to see the answer. Type **exit()** to close it.

COMMAND

```
>>> 5 + 3
8
>>> exit()
```

3. Jupyter Notebook

1. Install Python first.
2. Install Jupyter using pip: **pip install notebook**
3. Start it by typing: **jupyter notebook**. It opens in your browser.

Useful pip commands

pip is Python's tool for installing extra packages (libraries). Think of it as a **Play Store for Python**.

What you want to do	Command
See all installed packages	<code>pip list</code>
Check if one package is installed	<code>pip show packagename</code>
Install a package	<code>pip install packagename</code>
Update a package	<code>pip install --upgrade packagename</code>
Remove a package	<code>pip uninstall packagename</code>

4. Google Colab

Google Colab (<https://colab.research.google.com>) lets you run Python **in your browser**. You do not need to install anything. Just log in with your Google account. It is free and is very popular for Machine Learning.

5. PyCharm

PyCharm is a professional code editor. It helps you by completing code, showing mistakes and finding bugs. Download it from <https://www.jetbrains.com/pycharm/download/>

1.12 Virtual environment (venv and Miniconda)

Definition: A virtual environment is a separate folder for one project. It has its own Python and its own packages.

In simple words: Each project gets its own room, so the packages of one project do not disturb another project.

Example: Project A needs Django version 4 and Project B needs Django version 5. Both cannot be installed in the same place, so we make a separate environment for each project.

a) Using venv (comes with Python)

COMMAND

```
python -m venv projectA      # step 1: create
projectA\Scripts\activate    # step 2: start it (Windows)
source projectA/bin/activate # step 2: start it (Linux / Mac)
deactivate                   # step 3: stop it
```

When the environment is on, you will see its name at the start of the line, like **(projectA) D:\>**. You can also use the **virtualenv** package (**pip install virtualenv**), which works in a similar way.

b) Using Miniconda

Miniconda is a small installer that gives you Python and a tool called **conda** for making environments.

COMMAND

```
conda create --name projectA python=3.12 # create
conda activate projectA                  # start
conda install numpy                       # install a package
conda env list                           # see all environments
conda deactivate                          # stop
```

1.13 Checking code quality with pylint

pylint checks your code like a teacher checks an answer sheet. It points out mistakes and bad style, and gives your code **marks out of 10**.

COMMAND

```
pip install pylint
pylint add.py
```

OUTPUT

```
add.py:1:0: C0114: Missing module docstring (missing-module-docstring)
add.py:1:0: C0103: Constant name "a" doesn't conform to UPPER_CASE naming style
-----
Your code has been rated at 0.00/10
```

1.14 What is requirements.txt?

requirements.txt is a list of all the packages your project needs. When you share your project, the other person can install everything with one command.

In simple words: It is like a shopping list for your project.

Step 1 (your computer): save the list of packages into a file.

COMMAND

```
pip freeze > requirements.txt
```

The file will look like this:

CODE

```
numpy==2.1.0
pandas==2.2.2
Flask==3.0.3
```

Step 2 (other person's computer): install everything from the list.

COMMAND

```
pip install -r requirements.txt
```

1.15 Garbage Collector (gc)

Definition: The garbage collector automatically removes data from memory when the program no longer uses it.

In simple words: It works like a cleaner who throws away things nobody is using, so memory does not get full.

Python counts how many variables are using each value. This is called the **reference count**. When the count becomes **0**, the value is deleted from memory automatically.

EXAMPLE

```
import sys
import gc

a = [1, 2, 3]
b = a                # second reference to the same list
print(sys.getrefcount(a)) # +1 for the temporary argument

del b                # remove one reference
print(sys.getrefcount(a))

print(gc.isenabled()) # is automatic GC on?
print(gc.collect())   # run GC manually; returns objects collected
```

OUTPUT

```
3
2
True
0
```

The count shows 3 at first (a, b and one extra used by the function itself). After **del b** it becomes 2.

Quick Revision – Chapter 1

- Python is easy, free, and used for many kinds of work.

- Python is both compiled (to bytecode) and interpreted (by the PVM).
- Bytecode is stored in .pyc files inside the __pycache__ folder.
- In Python we do not write data types; use type() to check them.
- pip installs packages; requirements.txt lists the packages a project needs.
- A virtual environment keeps each project's packages separate.

2. Data Types and Operators

2.1 What is a variable?

Definition: A variable is a name given to a place in memory (RAM) where we store a value to use later.

In simple words: A variable is like a labelled box. The label is the name and the thing inside is the value.

EXAMPLE

```
num = 10
name = 'python'
print(num, name)
```

OUTPUT

```
10 python
```

Rules for giving names to variables

Rule	Correct	Wrong
Start with a letter or an underscore (_)	name, _total	1name
Use only letters, numbers and _	first_name	first name, first-name
Capital and small letters are different	age and Age are two variables	-
Do not use Python keywords	my_class	class, if, for

2.2 Ways to give values to variables

a) One value to many variables

EXAMPLE

```
a = b = c = 100
print(a, b, c)
```

OUTPUT

```
100 100 100
```

b) Different values to many variables in one line

EXAMPLE

```
x, y, z = 10, 20.5, "Pune"
print(x, y, z)
```

OUTPUT

```
10 20.5 Pune
```

Tip: You can swap two values in one line: `a, b = b, a`

2.3 What is a data type?

A data type tells **what kind of value** is stored in a variable: a whole number, a decimal number, text, a list, and so on.

In C or Java, we must write the type first, and the computer keeps that much memory ready:

CODE

```
int num = 10;           // usually 4 bytes
float salary = 455.50; // 4 bytes
```

In Python, we just write the value. Python decides the type by itself:

CODE

```
num = 30           # Python decides that num is an int
```

Data types in Python

Data type	What it stores	Example
int	Whole numbers	age = 25
float	Decimal numbers	price = 45.50
complex	Numbers with a real and an imaginary part	z = 3 + 4j
bool	Only True or False	passed = True
str	Text, written inside quotes	name = "Python"
list	Many values in []; can be changed	marks = [80, 75, 90]
tuple	Many values in (); cannot be changed	point = (10, 20)
dict	Key : value pairs in { }	d = {"name": "Ajinkya"}
set	Unique values in { }; no duplicates	s = {1, 2, 3}
frozenset	A set that cannot be changed	fs = frozenset({1, 2, 3})
None	Means "no value" or "empty"	result = None

Let us check the type of each value using a loop:

EXAMPLE

```
values = [10, 45.50, 3 + 4j, True, "Python", [1, 2, 3],
          (1, 2, 3), {"name": "Ajinkya"}, {1, 2, 3},
          frozenset({1, 2, 3}), None]

for v in values:
    print(v, "->", type(v))
```

OUTPUT

```

10 -> <class 'int'>
45.5 -> <class 'float'>
(3+4j) -> <class 'complex'>
True -> <class 'bool'>
Python -> <class 'str'>
[1, 2, 3] -> <class 'list'>
(1, 2, 3) -> <class 'tuple'>
{'name': 'Ajinkya'} -> <class 'dict'>
{1, 2, 3} -> <class 'set'>
frozenset({1, 2, 3}) -> <class 'frozenset'>
None -> <class 'NoneType'>

```

2.4 What are operators?

Definition: An operator is a symbol that does some work (an operation) on values.

CODE

```
result = a + b
```

Here, **a** and **b** are called **operands** (the values), and **+** is the **operator** (the work to be done).

Python has 7 types of operators:

1. Arithmetic operators (maths)
2. Relational operators (compare)
3. Logical operators (and, or, not)
4. Assignment operators (store and update)
5. Bitwise operators (work on 0s and 1s)
6. Membership operators (in, not in)
7. Identity operators (is, is not)

a) Arithmetic operators – for maths

Let us take a = 17 and b = 5.

Operator	Name	Example	Result
+	Addition	a + b	22
-	Subtraction	a - b	12
*	Multiplication	a * b	85
/	Division (answer is always a decimal)	a / b	3.4
//	Floor division (answer without the decimal part)	a // b	3
%	Modulus (gives the remainder)	a % b	2
**	Power	a ** b	1419857

EXAMPLE

```
a, b = 17, 5
print(a / b)      # 3.4
print(a // b)     # 3
print(a % b)      # 2
print(-17 // 5)   # -4 (floor rounds towards minus infinity)
```

OUTPUT

```
3.4
3
2
-4
```

Note: `//` always rounds **down** to the smaller number. So `-17 // 5` is `-4` (not `-3`), because `-4` is smaller than `-3.4`.

Tip: Use `% 2` to check even or odd numbers. If `num % 2` is 0, the number is even.

b) Relational operators – for comparing

These operators compare two values. The answer is always **True** or **False**. Let us take `a = 10` and `b = 20`.

Operator	Meaning	Example	Result
<code>==</code>	Equal to	<code>a == b</code>	False
<code>!=</code>	Not equal to	<code>a != b</code>	True
<code>></code>	Greater than	<code>a > b</code>	False
<code><</code>	Less than	<code>a < b</code>	True
<code>>=</code>	Greater than or equal to	<code>a >= 10</code>	True
<code><=</code>	Less than or equal to	<code>b <= 15</code>	False

Note: `=` is used to store a value. `==` is used to compare two values. Do not mix them up.

c) Logical operators – and, or, not

These operators join two or more conditions. Python uses the English words **and**, **or**, **not**. (The symbols `&&`, `||` and `!` from C and Java do not work in Python.)

and – the answer is True only when **both** conditions are True.

Condition 1	Condition 2	Result of and
False	False	False
True	False	False
False	True	False
True	True	True

or – the answer is True when **at least one** condition is True.

Condition 1	Condition 2	Result of or
False	False	False
True	False	True
False	True	True
True	True	True

not – turns True into False, and False into True.

EXAMPLE

```
age = 22
has_id = True

print(age >= 18 and has_id)    # both True
print(age < 18 or has_id)      # one True
print(not has_id)
```

OUTPUT

```
True
True
False
```

In simple words: and = “both must be true”, or = “any one is enough”, not = “the opposite”.

d) Assignment operators – store and update values

These operators store a value in a variable or change the value already stored. Python has **no ++ or -- operators** (like `i++`). Write `i += 1` instead.

Operator	Example	Same as	Meaning
=	<code>x = 10</code>	<code>x = 10</code>	Store 10 in x
+=	<code>x += 5</code>	<code>x = x + 5</code>	Add 5 to x
-=	<code>x -= 5</code>	<code>x = x - 5</code>	Subtract 5 from x
*=	<code>x *= 5</code>	<code>x = x * 5</code>	Multiply x by 5
/=	<code>x /= 5</code>	<code>x = x / 5</code>	Divide x by 5
%=	<code>x %= 5</code>	<code>x = x % 5</code>	Keep the remainder
**=	<code>x **= 2</code>	<code>x = x ** 2</code>	Square x
//=	<code>x //= 3</code>	<code>x = x // 3</code>	Floor divide x by 3

EXAMPLE

```
x = 10
x += 5
print(x)
x *= 2
print(x)
x //= 4
print(x)
```

OUTPUT

```
15
30
7
```

e) Bitwise operators – work on 0s and 1s

Computers store numbers in **binary** (only 0 and 1). Bitwise operators work on these 0s and 1s. Let us take a = 12 (binary **1100**) and b = 10 (binary **1010**).

Operator	Name and rule	Working	Result
&	AND – 1 only if both bits are 1	1100 & 1010 = 1000	8
	OR – 1 if any bit is 1	1100 1010 = 1110	14
^	XOR – 1 if the bits are different	1100 ^ 1010 = 0110	6
~	NOT – answer is -(a + 1)	~12 = -(12 + 1)	-13
<<	Left shift – multiply by 2 for each shift	12 << 2 = 12 × 4	48
>>	Right shift – divide by 2 for each shift	12 >> 2 = 12 ÷ 4	3

EXAMPLE

```
a, b = 12, 10
print(a & b, a | b, a ^ b, ~a, a << 2, a >> 2)
print(bin(a), bin(b))
```

OUTPUT

```
8 14 6 -13 48 3
0b1100 0b1010
```

Tip: The `bin()` function shows the binary form of a number. The `0b` at the start means “binary”.

f) Membership operators – in, not in

These operators check whether a value is **present** inside a string, list, tuple, set or dictionary. The answer is True or False.

- **in** – True if the value is present.
- **not in** – True if the value is not present.

EXAMPLE

```
languages = ["Python", "Java", "C"]
print("Python" in languages)
print("Ruby" not in languages)
print("y" in "Python")

student = {"name": "Riya", "age": 21}
print("name" in student)      # dict checks keys
```

OUTPUT

```
True
True
True
True
```

g) Identity operators – is, is not

These operators check whether two variables point to the **same object in memory**, not only whether they have the same value.

- **is** – True if both are the same object.
- **is not** – True if they are different objects.

EXAMPLE

```
a = [1, 2, 3]
b = a          # b refers to the same list
c = [1, 2, 3]  # c is a new list with equal values

print(a is b)
print(a is c)
print(a == c)  # == compares values, not addresses
print(a is not c)
```

OUTPUT

```
True
False
True
True
```

In simple words: Two students can have the same name (== is True) but they are still two different people (is is False).

Order of operators (precedence)

When one line has many operators, Python follows a fixed order, just like the BODMAS rule in maths:

1. Brackets ()
2. Power **
3. Multiply and divide: * / // %
4. Add and subtract: + -
5. Comparisons: == != > < >= <=
6. Logical: not, then and, then or

EXAMPLE

```
print(2 + 3 * 4 ** 2)    # 4**2 = 16, 3*16 = 48, 2+48 = 50
```

OUTPUT

```
50
```

Quick Revision – Chapter 2

- A variable is a named box that stores a value.
- Main data types: int, float, complex, bool, str, list, tuple, dict, set, frozenset, None.
- / gives a decimal answer, // removes the decimal part, % gives the remainder.

- Comparison operators always give True or False.
- Python uses `and`, `or`, `not` – not `&&`, `||`, `!`.
- Python has no `++`; use `+= 1`.
- `==` compares values; `is` checks whether it is the same object.

3. Conditional Statements (Decision Making)

In daily life we make decisions: **if** it rains, take an umbrella; **else**, wear sunglasses. In the same way, conditional statements let a program **choose what to do based on a condition**.

Note: Python uses **indentation** (4 spaces at the start of a line) to show which lines belong to an if block. Wrong spacing gives an error.

Python has 5 kinds of conditional statements:

1. Simple if
2. if – else
3. if – elif – else
4. Nested if – else
5. match – case

3.1 Simple if

Checks **one condition**. If the condition is True, the code inside runs. If it is False, nothing happens.

SYNTAX

```
if condition:
    statements
```

EXAMPLE

```
marks = 75
if marks >= 40:
    print("You have passed.")
```

OUTPUT

```
You have passed.
```

3.2 if – else

Checks **one condition** and has two paths. If the condition is True, the **if** part runs. Otherwise, the **else** part runs.

SYNTAX

```
if condition:
    statements      # True block
else:
    statements      # False block
```

EXAMPLE

```
num = 7
if num % 2 == 0:
    print(num, "is even")
else:
    print(num, "is odd")
```

OUTPUT

```
7 is odd
```

3.3 if – elif – else

Used when there are **many conditions**. Python checks them from top to bottom. The **first** condition that is True runs, and the rest are skipped. If none are True, the **else** part runs.

SYNTAX

```
if condition1:
    statements
elif condition2:
    statements
elif condition3:
    statements
else:
    statements
```

EXAMPLE

```
marks = 68

if marks >= 75:
    grade = "Distinction"
elif marks >= 60:
    grade = "First Class"
elif marks >= 40:
    grade = "Pass"
else:
    grade = "Fail"

print("Grade:", grade)
```

OUTPUT

```
Grade: First Class
```

Here, 68 is not ≥ 75 , so Python goes to the next condition. 68 is ≥ 60 , so the grade is “First Class”, and Python stops checking.

3.4 Nested if – else

An if statement **inside another** if statement. The inner condition is checked only if the outer condition is True.

SYNTAX

```
if condition1:
    if condition2:
        statements
    else:
        statements
else:
    statements
```

EXAMPLE

```
age = 20
has_licence = False

if age >= 18:
    if has_licence:
        print("You can drive.")
    else:
        print("Apply for a driving licence first.")
else:
    print("You are not old enough to drive.")
```

OUTPUT

```
Apply for a driving licence first.
```

3.5 match – case (Python 3.10 and newer)

Used when we compare **one value with many options** and want to run only the option that matches. It is like **switch – case** in C or Java.

case _: is the **default** case. It runs when no other case matches.

SYNTAX

```
match variable:
    case value1:
        statements
    case value2:
        statements
    case _:
        statements      # default
```

EXAMPLE

```
day = 3

match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 6 | 7:
        print("Weekend")      # | matches either value
    case _:
        print("Invalid day")
```

OUTPUT

```
Wednesday
```

Note: In C or Java, switch needs **break**. In Python, match – case does **not** need **break**; only the matching case runs.

3.6 Keywords in Python

Keywords are special words that Python has reserved for its own use. **We cannot use them as variable names.**

Python 3.12 has 35 keywords. The words **match** and **case** are special: they act like keywords only inside a match statement.

CODE

```
import keyword
print(keyword.kwlist)
```

False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield

Quick Revision – Chapter 3

- **if** – one condition, one action.
- **if – else** – one condition, two paths.
- **if – elif – else** – many conditions; only the first True one runs.
- **Nested if** – an if inside another if.
- **match – case** – compare one value with many options; **case _** is the default.
- Indentation (4 spaces) is compulsory in Python.